

The background features a stylized illustration of a code editor interface on the left, showing a file explorer, a search bar, and a code editor with syntax-highlighted text. To the right of the code editor are architectural diagrams, including a cross-section of a building with a roof and a compass rose. The entire scene is set against a dark blue background with a light green grid pattern.

Praktikum 11: Membangun Frontend Interaktif dengan VueJS 3

Integrasi REST API menggunakan Axios dan
Reactive Data Binding

Tiga Pilar Kompetensi

Konsep API

Memahami dasar komunikasi data melalui REST API.

Framework VueJS

Menguasai arsitektur Component-based dan Reactive Data Binding pada VueJS 3.

Implementasi Frontend

Merakit aplikasi antarmuka utuh yang mampu mengelola data secara dinamis dari API.

Anatomi VueJS: Reaktivitas Data

State Layer

```
data: {  
  name: 'John Doe',  
  age: 30,  
  isActive: true  
}
```

Variabel JavaScript yang menyimpan data aplikasi.

Two-Way Street

v-model / Reactivity

View Layer

```
<input v-model="name"  
  placeholder="Enter name">  
<p>Hello, {{ name }}!</p>
```

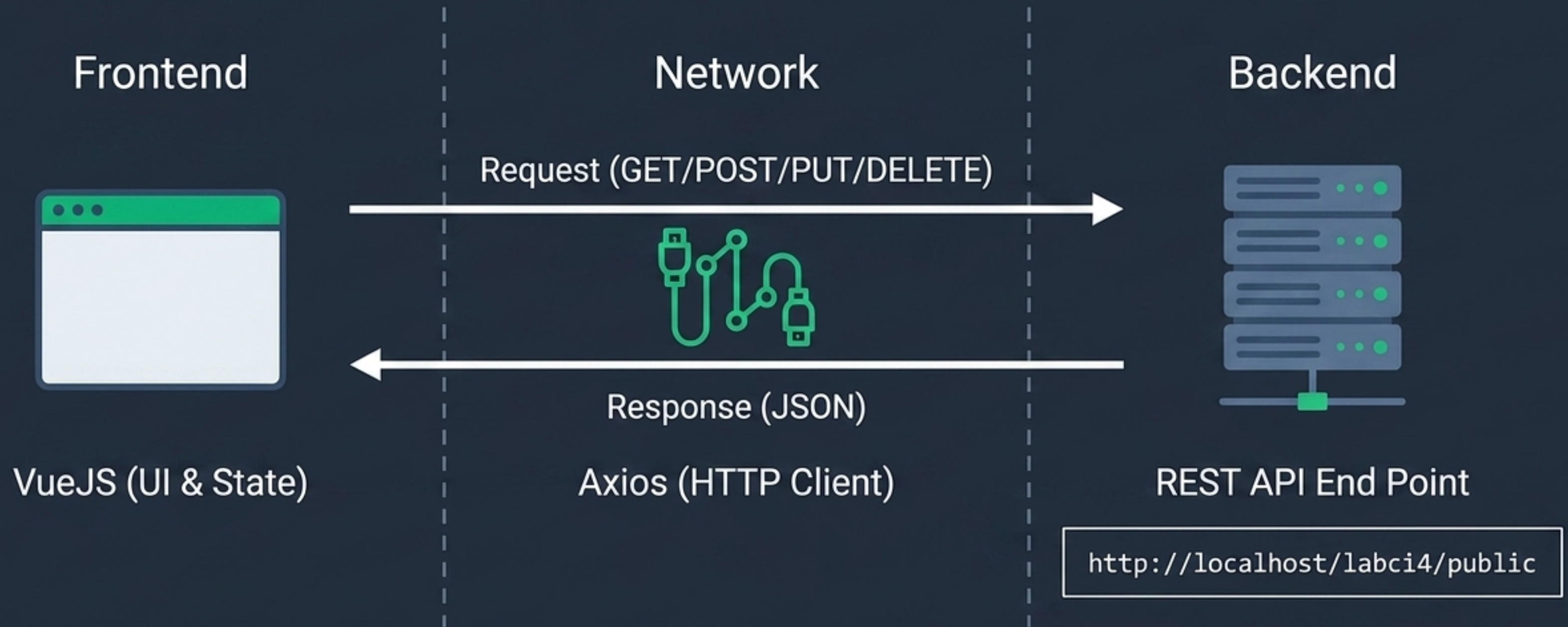
John Doe

Hello, John Doe!

Elemen HTML yang merender antarmuka pengguna.

Perubahan pada data (kiri) secara otomatis memperbarui **antarmuka (kanan)** tanpa memanipulasi DOM secara manual. Inilah inti dari Reactive Data Binding.

Peta Arsitektur Sistem Komunikasi



VueJS fokus murni pada antarmuka, sementara Axios bertugas mengambil dan mengirim data ke server Backend.

Persiapan Workspace & Library

Struktur Direktori

```
|— index.html
|— assets
|   |— css
|   |   |— style.css
|   |— js
|       |— app.js
```

Integrasi via CDN

Library Inti VueJS 3

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js">
</script>
```

Library Axios untuk pemanggilan API

```
<script src="https://unpkg.com/axios/dist/axios.min.js">
</script>
```


Tahap 1: Iterasi Antarmuka (HTML)

```
<tr v-for="(row,index) in artikel">  
  <td>{{ row.judul }}</td>  
  <td>{{ statusText(row.status) }}</td>  
</tr>
```

v-for: Direktif looping otomatis. Membaca setiap item di dalam array `artikel`.

row.judul

row.judul

row.judul

row.judul

v-for: Direktif looping otomatis. Membaca setiap item di dalam array `artikel`.

{{ ... }}: Sintaks Mustache untuk menampilkan nilai variabel dari JavaScript ke dalam HTML.

Tahap 2: Menarik Data (Vue & Axios)

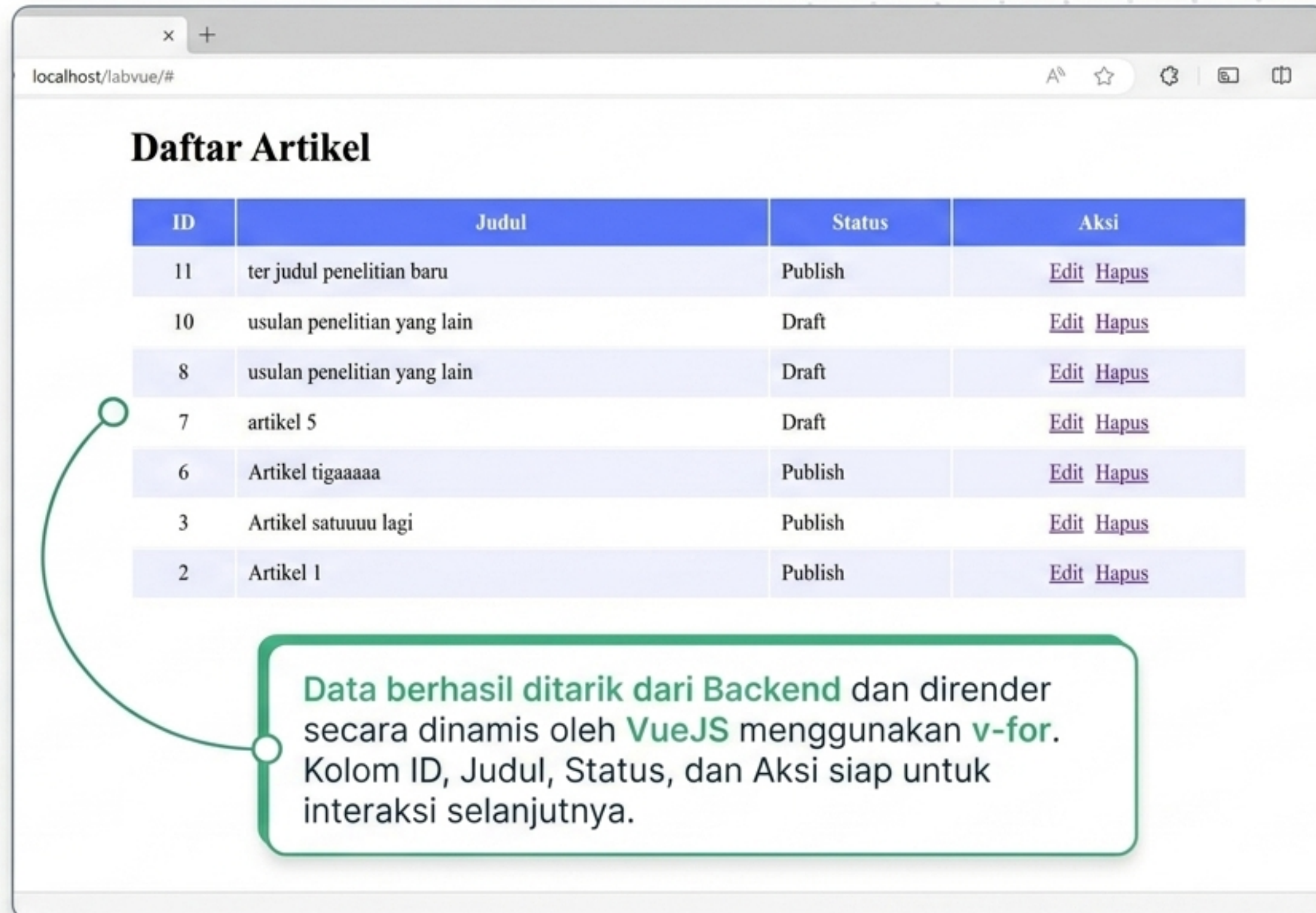
```
mounted() {  
  this.loadData()  
},  
methods: {  
  loadData() {  
    axios.get(apiUrl + '/post')  
      .then(response => {  
        this.artikel = response.data.artikel  
      })  
  }  
}
```

1. `mounted()`: Memicu aksi saat aplikasi pertama kali dimuat.

2. `axios.get()`: Mengirim permintaan HTTP GET ke REST API.

3. `this.artikel = ...`: Mengisi variabel Vue dengan data JSON dari server. (Reaktivitas otomatis mengupdate tabel HTML!).

Checkpoint 1: Render Daftar Artikel



The screenshot shows a web browser window with the address bar displaying 'localhost/labvue/#'. The page title is 'Daftar Artikel'. Below the title is a table with 4 columns: ID, Judul, Status, and Aksi. The table contains 7 rows of data. A green callout box with a circular arrow points to the table, containing the following text:

Data berhasil ditarik dari Backend dan dirender secara dinamis oleh VueJS menggunakan v-for. Kolom ID, Judul, Status, dan Aksi siap untuk interaksi selanjutnya.

ID	Judul	Status	Aksi
11	ter judul penelitian baru	Publish	Edit Hapus
10	usulan penelitian yang lain	Draft	Edit Hapus
8	usulan penelitian yang lain	Draft	Edit Hapus
7	artikel 5	Draft	Edit Hapus
6	Artikel tigaaaaa	Publish	Edit Hapus
3	Artikel satuuuu lagi	Publish	Edit Hapus
2	Artikel 1	Publish	Edit Hapus

Tahap 3: Merancang Modal Form

```
index.html x
<div class="modal" v-if="showForm">
  <div class="modal-content">
    <input type="text" v-model="formData.judul">
    <h3>input type="">{{ fomTitle }}</h3>
    <div>textarea name="isi" id="isi"
rows=""<textarea></div>
    <div>
      <select name="status" id="status">
        <option v-for="option in statusOptions">
          {{ option.text }}
        </option>
      </select>
    </div>
    <input type="hidden" id="id" v-model="formData.id">
    <button type="submit" id="btnSimpan">Simpan</button>
    <button @click="showForm = false">Batal</button>
  </form>
</div>
</div>
```

State Map



showForm



State: showForm = false
→ Modal Tersembunyi.



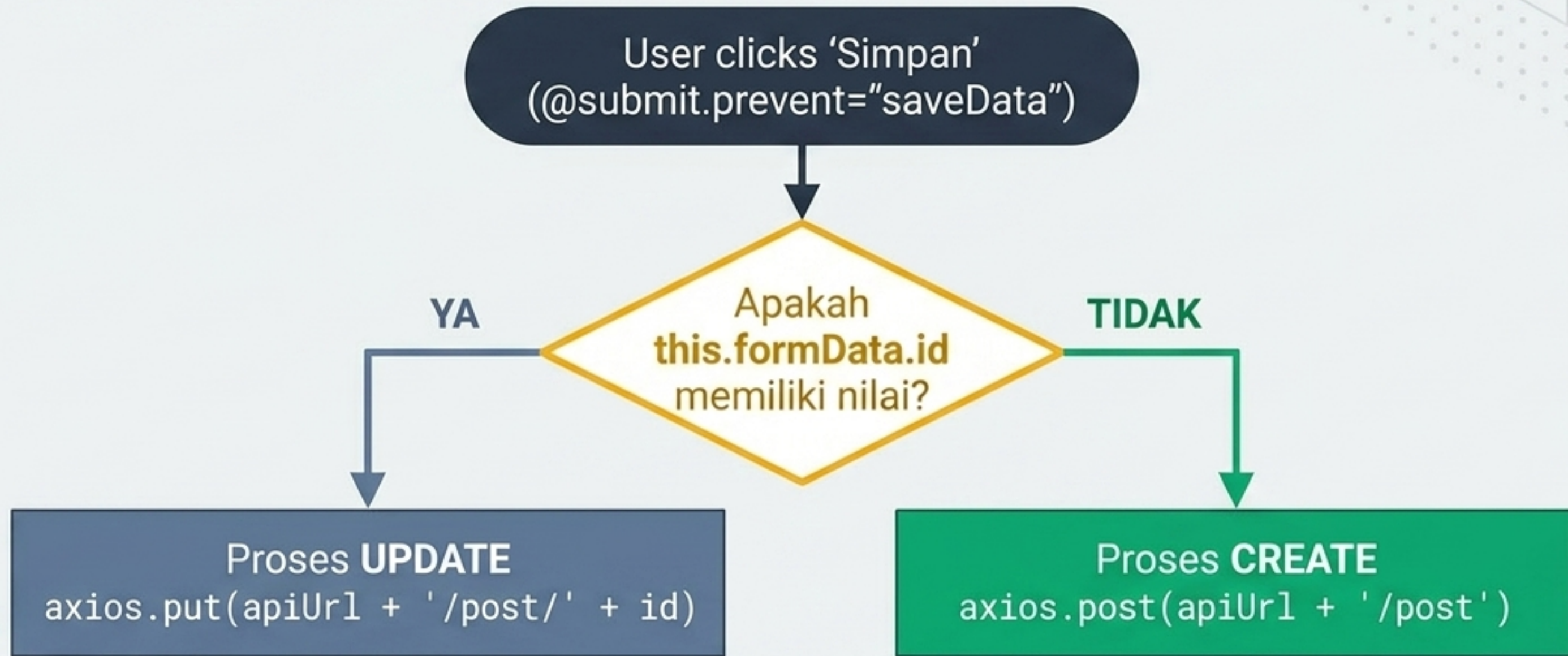
State: showForm = true
→ Modal Muncul (Dirender oleh DOM).

Direktif `v-model` mengikat nilai input form secara real-time langsung ke dalam objek `formData` di JavaScript.

Tahap 4: Mengelola State Form (Tambah vs Ubah)

Logika Tambah - Metode tambah()	Logika Ubah - Metode edit(data)
Aksi: Mengosongkan objek formData.	Aksi: Menyalin data dari baris tabel ke dalam formData.
Hasil State: id: null judul: '' status: 0	Hasil State: id: data.id judul: data.judul status: data.status
Kedua metode akan diakhiri dengan <code>this.showForm = true</code> untuk memunculkan modal.	

Matriks Integrasi API (Logika Simpan)



VueJS cukup cerdas untuk menggunakan satu form yang sama untuk dua operasi berbeda, hanya dengan memeriksa eksistensi id.

Tahap 5: Eksekusi Hapus Data

```
1  if (confirm('Yakin menghapus data?')) {  
2    axios.delete(apiUrl + '/post/' + id)  
3      .then(response => {  
4        this.artikel.splice(index, 1)  
5      })  
6  }
```

Delete

- 1 — Konfirmasi: `confirm('Yakin menghapus data?')` mencegah penghapusan tidak disengaja.
- 2 — Permintaan API: `axios.delete(...)` mengirim perintah ke Backend.
- 3 — Pembaruan Reaktif (Kunci): **`this.artikel.splice(index, 1).`**

⚠ Alih-alih melakukan reload halaman atau memanggil `loadData()` ulang, metode **`splice`** langsung menghapus elemen dari array Vue. Antarmuka otomatis merespons seketika!

Sentuhan Akhir (Styling CSS)

Judul	Status	Aksi
Artikel 1	Publish	Edit Hapus
Artikel 2	Draft	Edit Hapus



Modal Overlay: Menggunakan `position: fixed` dan `background-color: rgba(...)` untuk menciptakan efek pop-up di atas konten utama.

Hapus Data ×

Yakin menghapus data?

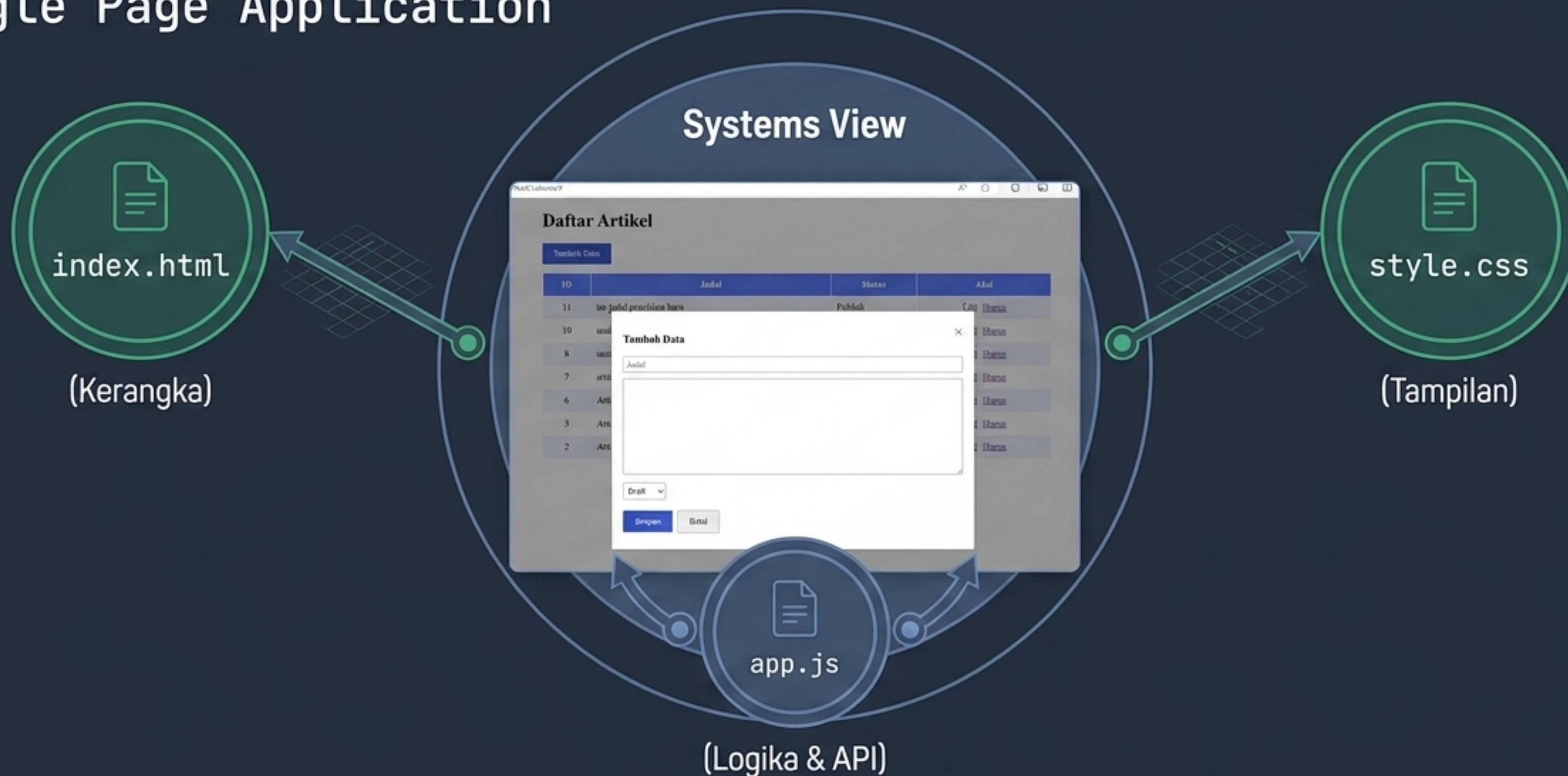
Batal Hapus

Judul	Status	Aksi
Artikel 1	Publish	Edit Hapus
Artikel 2	Draft	Edit Hapus

Zebra Striping: Menggunakan pseudo-class `tr:nth-child(odd)` untuk membedakan warna baris tabel.

CSS mengubah struktur kerangka (HTML) menjadi pengalaman visual yang profesional dan mudah dibaca.

Sintesis Aplikasi: Single Page Application



Melalui kombinasi reaktivitas VueJS, komunikasi asinkron Axios, dan gaya visual CSS, kita telah membangun sebuah antarmuka yang mengelola data secara dinamis tanpa pernah memuat ulang halaman (page reload). Inilah fundamental dari web modern.

Tugas Laporan Praktikum

- ☐ Melanjutkan praktikum pada repositori Lab11Web_VueJS.
- ☐ Kerjakan seluruh latihan dari Tahap 1 hingga Tahap 5 secara berurutan.
- ☐ Ambil Screenshot pada setiap perubahan antarmuka.
- ☐ Perbarui file README.md dengan penjelasan instruktif dan lampirkan screenshot.
- ☐ Commit dan Push hasil ke repositori GitHub.
- ☐ Kumpulkan URL repositori melalui e-learning ecampus.

Anda bebas melakukan improvisasi desain maupun fitur.